

HACETTEPE UNIVERSITY DEPARTMENT OF COMPUTER ENGINEERING
BBM384 SOFTWARE ENGINEERING LABORATORY
EXPERIMENT SHEET
2024 Spring

Instructors:

Assoc. Prof. Dr. Ayça KOLUKISA, Assoc. Prof. Dr. Ebru GÖKALP, Dr. Tuğba ERDOĞAN

Teaching Assistants:

R. A. Bahar GEZİCİ, R.A. Sibel KAPAN, R.A. Aslı Mükerrerem TAŞGETİREN

1. Purpose and Scope

The purpose of this experiment sheet is to guide students who enroll in the BBM384 course in developing their laboratory projects. The projects are pre-defined and the process defined in this document is applied through the development of the projects. While applying the process, the students follow the schedule given by the instructors at the beginning of the course.

2. Abbreviations

SDLC : Software Development Life Cycle
SRS : Software Requirements Specification
TCD : Test Case Definition
SDD : Software Design Description
STR : Software Test Report
GUI : Graphical User Interface

3. Roles and Responsibilities

Roles	Responsibilities
Student	Responsible for conforming to the requirements of this process, and for timely generation of the outputs of this process.
Teaching Assistant(s)	Responsible for timely evaluation and feedback of the outputs generated by the students.
Instructor(s)	Responsible for resolution of issues likely to occur between teaching assistants and the students.
Stakeholder	Any person that has a relation with the system to be developed (including students, teaching assistants, and instructors).
Team Member	A student that takes a role in the development of the system. Undertaken roles include Software Project Manager, Software Analyst, Software Architect, Software Developer, Software Change/Configuration Manager, and Software Tester. Each student must have <u>Software Developer</u> role during development. A student may have more than one role during development.

4. Inputs

- (i) Vision Document Template
- (ii) Project Plan Template
- (iii) Software Requirements Specification Template
- (iv) Use Case Definition Template
- (v) Test Case Definition Template
- (vi) Graphical User Interface Design Template
- (vii) Architecture Notebook Template
- (viii) Risk Management Report Template
- (ix) Configuration/Change Management Report Template
- (x) Software Design Description Template
- (xi) Software Coding Standard
- (xii) Test Script Template
- (xiii) Software Test Report Template
- (xiv) GithubFlow: <https://guides.github.com/introduction/flow/>

NOTE:

- Risk Management Report will be created by using the lists on the page:
https://docs.google.com/spreadsheets/d/1kJ_xwoGB3HXMxik8cqvt273BMKKofUHd/edit#gid=768757863
- Configuration/Change Management Report will be created by using the lists on the page:
https://docs.google.com/spreadsheets/d/1kJ_xwoGB3HXMxik8cqvt273BMKKofUHd/edit#gid=9565548

5. Outputs

- (i) Software Vision Document
- (ii) Software Project Plan
- (iii) Software Requirements Specification(SRS)
- (iv) Use Case Definitions (as an attachment to SRS)
- (v) Graphical User Interface Definitions (as an attachment to SRS)
- (vi) Test Case Definitions (as an attachment to SRS)
- (vii) Architecture Notebook
- (viii) Risk Management Report
- (ix) Configuration/Change Management Report
- (x) Software Design Document
- (xi) Software Code (and Software Coding Standard if modified)
- (xii) Software Test Report (STR)
- (xiii) Test Case and Test Script Definitions (as an attachment to STR)
- (xiv) Project's Github Repository (master and member branches, issues, pull requests, commits etc.)

6. System to Be Developed

Imagine you have been assigned to develop an online review and recommendation system for a popular e-commerce platform named ShopSmart, called Review and Recommendation System for ShopSmart (RRSS). This system aims to enhance the shopping experience for users by providing them with valuable insights from fellow shoppers and personalized product recommendations. This system will be an ideal system for administrator, user, merchant, and community moderator modules.

In RRSS for ShopSmart, users can submit reviews and ratings for products they have purchased, helping others make informed purchasing decisions. Additionally, the system utilizes advanced algorithms to analyze user preferences and behavior, generating personalized product recommendations tailored to each user's interests and browsing history.

Privacy and data security are paramount in RRSS for ShopSmart, with users' personal information and purchasing history kept confidential. Reviews are displayed anonymously, and users have control over their privacy settings, ensuring a secure and trusted shopping environment.

ShopSmart administrators can access aggregated review data and recommendation algorithms to gain insights into product performance, customer satisfaction, and emerging trends. This information can be used to optimize product listings, marketing strategies, and inventory management, ultimately driving sales and customer loyalty.

RRSS for ShopSmart will feature a user-friendly interface, allowing shoppers to easily submit reviews, browse product recommendations, and interact with other users. The system will be designed with scalability and extensibility in mind, adhering to the Open-Closed Principle to accommodate future enhancements and integrations as ShopSmart continues to grow and evolve in the competitive e-commerce market.

All software development activities will be done as a GitHub project. GitHub is the best way to build software together. Each member must create a branch where s/he can safely experiment and make changes by the following [GitHub flow](#). S/he will use a pull request to get feedback on his/her changes. After the changes are reviewed, s/he will merge them into the master branch and deploy his/her code to the project.

System Modules: The system modules for an online review and recommendation system (RRSS) are explained below:

Administrator Module:

- Logins to the system by entering his/her login id and password.
- Manages his/her account and member accounts. For example, s/he can send new passwords to the user who forgot his/her password.
- This module is responsible for overseeing the overall management and configuration of the review and recommendation system.
- Administrators have broader privileges, including the ability to manage user and merchant accounts, review content moderation, and enforce platform policies. They can customize review forms, set review parameters, and configure recommendation algorithms.
- They can monitor user activity, detect fraudulent behavior, and ensure the integrity of the review and recommendation ecosystem.
- Additionally, administrators may implement new features, conduct system updates, and handle escalated customer support issues.

User Module:

- Enrolls to the system and sends enrollment requests to the administrator.
- Logins to the system by entering his/her login id and password.
- Sends forgot password request when s/he forgets her/his password.
- Accesses and manages his/her account and can change their personal information. For example, s/he can add/delete/update her/his picture and change her/his password etc.
- Submits reviews and ratings for products they have purchased, contributing to the platform's collective knowledge base.
- Accesses to personalized product recommendations based on their browsing history, purchase behavior, and preferences.
- Views and interacts with reviews submitted by other users, helping them make informed purchasing decisions.
- Can share their reviews and recommended products on social media platforms, expanding the reach and visibility of their contributions.
- Bookmarks or saves products they are interested in for future reference, creating personalized wish lists within the platform.
- Engages with other members of the community through forums, discussion boards, or user-generated content sections, fostering a sense of belonging and collaboration.

There are two types of users (Registered and non-registered). Difference between registered and non-registered users are described below:

Registered Users: These users have created accounts on the platform and logged in. They can access all features and functionalities within the User Module, such as submitting reviews, browsing recommendations, engaging in community discussions, and managing their profiles.

Non-Registered Users: Non-registered users can access basic features of the User Module, such as browsing products and reading reviews. However, they cannot submit reviews, engage in community activities, or access personalized recommendations until they create an account and log in.

The User module may incorporate a reward system where users earn points or discounts for active participation, such as submitting high-quality reviews, receiving upvotes, or contributing valuable insights.

Merchant Module:

- Enrolls to the system and sends enrollment requests to the admin.
- Logins to the system by entering his/her login id and password.
- Sends forgot password request when s/he forgets her/his password.
- Accesses and manages his/her account and can change their personal information. E.g., s/he can add/delete/update her/his picture and change her/his password etc.
- Monitors and responds to customer reviews, addressing any concerns or feedback directly. The module facilitates direct communication between merchants and users, allowing merchants to respond to inquiries, address concerns, and provide personalized assistance.
- Accesses to analytics and insights related to their products' performance, including ratings, reviews, and sales data.

- Manages their product listings, including adding new products, updating descriptions, and adjusting pricing and inventory levels.
- Accesses to promotional tools, such as discount codes, coupons, etc.

Community Engagement Module:

The Community Engagement Module adds a social dimension to the platform, empowering users to interact, share knowledge, and build relationships with like-minded individuals within the online community. By fostering a sense of belonging and collaboration, this module enhances user engagement and loyalty, ultimately contributing to the platform's success and growth.

In the Community Engagement Module, the primary actor is the **User Community**. This includes all registered users of the platform who actively participate in community discussions, contribute user-generated content, and interact with other members of the community, and non-registered users who have limited access to the Community Engagement Module. They may be able to view discussions and user-generated content but cannot actively participate or contribute. However, non-registered users may be encouraged to register for an account to unlock full access to the community features and benefits, such as posting comments, earning rewards, and connecting with other users. The platform may provide prompts or incentives for non-registered users to sign up, such as offering exclusive content, discounts, or access to premium features upon registration.

Additionally, **Community Moderators** may also act within this module. They are responsible for overseeing community discussions, ensuring adherence to community guidelines, and facilitating a positive and inclusive environment within the forums and user-generated content sections. Community Moderators may have elevated permissions to moderate content, resolve disputes, and enforce community standards.

Both the User Community and Community Moderators play pivotal roles in driving engagement, fostering collaboration and supportive online community within the platform's Community Engagement Module.

- *Discussion Forums*: This module facilitates discussion forums or community boards where users can engage in discussions, ask questions, and share experiences related to products, brands, or shopping tips.
- *User-generated Content*: Users can contribute user-generated content, such as blog posts, product tutorials, or style guides, to enrich the platform's content and provide valuable insights to fellow shoppers.
- *Expert Q&A*: Users can seek advice and expertise from industry experts or influencers through a dedicated Q&A section, where they can pose questions and receive personalized responses or recommendations.
- *Local Communities*: The module allows users to connect with others in their local area or with similar interests, facilitating local meetups, group purchases, or community events organized by users or merchants.

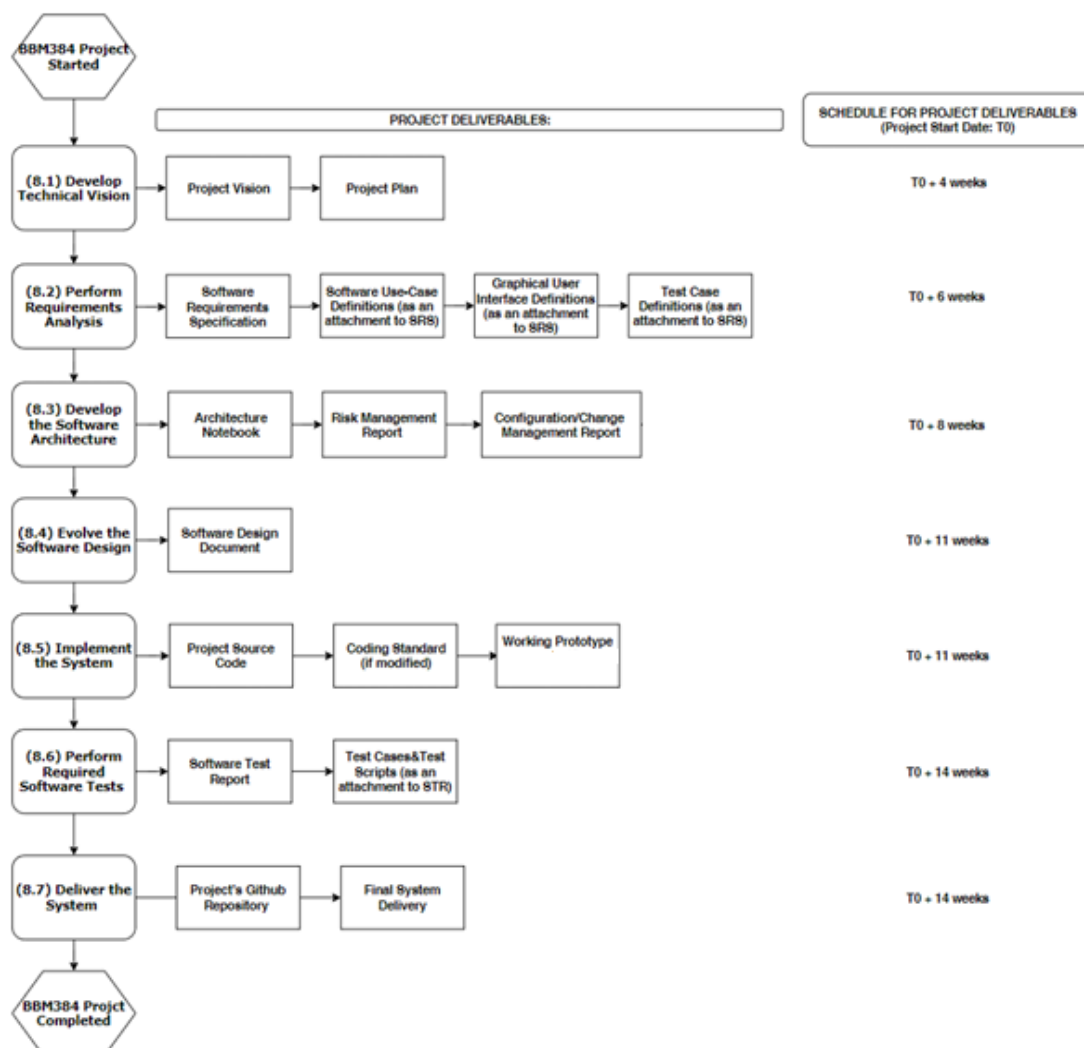
These modules collectively form the backbone of the online review and recommendation system, empowering users, merchants, community moderators, and administrators to engage with the platform effectively. Each module serves a specific function, contributing to the overall success and functionality of the system in facilitating informed purchasing decisions and fostering a trustworthy online shopping environment.

NOTE: You **MUST add** additional functional requirements different from the described above and **MUST add** non-functional requirements.

System Development Constraints

- The system to develop can be either a web application or a mobile application - not both!
- There must be minimum 9 and maximum 12 critical use-cases in your system to develop.
- Each group must follow GitHub Flow to keep change and configuration management activities: <https://guides.github.com/introduction/flow/>
 - Each group member must create own branch, get feedback for its own code by creating pull requests, and deploy own code to master branch.
- Model-View-Controller (MVC) architectural pattern must be used for this system.
- JAVA or C# programming language must be used to develop the system.
- For frontend development, groups can use some frameworks such as React, Angular etc.
- Groups can use any database management system such as Oracle, MySQL, and SQL Server. You are not allowed to use Firebase for database.
- JAVA or C# programming language must be used to provide connection between the database and the software code.

7. Process Flow



8. Process Activities

8.1. Develop Technical Vision & Project Plan

This activity includes the following tasks.

(i) Develop Technical Vision

The students define the vision for the future system by describing the problem and features based on stakeholder requests. The students document the vision as conformant to [Vision Document Template](#).

(ii) Develop Project Plan

The students plan the system development by describing the content of the system, project objectives, project organization, development parameters (effort and cost), development process steps, and project milestones. The project plan is documented as conformant to [Project Plan Template](#).

8.2. Perform Requirements Analysis

The students analyze the requirements of the system to be developed. The students apply use-case analysis to identify the requirements of the system and document the requirements as conformant to [Software Requirements Specification Template](#).

The students document use cases as conformant to [Use Case Definition Template](#) or draw activity diagram for each use case. The students also define graphical user interfaces that the users will interact while executing use cases, as conformant to [Graphical User Interface Design Template](#). The students will submit both documents as an attachment to SRS.

At the end of the analysis, the students also define test cases and data for critical use-case scenarios as basis for functional and acceptance tests. The students document test cases as conformant to [Test Case Definition Template](#) and submit the document as an attachment to SRS.

This activity includes the following tasks.

(i) Identify and Outline Requirements

This task describes how to identify and outline the requirements for the system so that the scope of work can be determined.

The purpose of this task is to identify and capture functional and non-functional requirements for the system. These requirements form the basis of communication and agreement between the stakeholders and the development team on what the system must do to satisfy stakeholder needs. The goal is to understand the requirements at a high-level so that the initial scope of work can be determined.

(ii) Detail Use Case Scenarios

This task describes how to detail use-case scenarios for the system.

The purpose of this task is to describe use-case scenarios in sufficient detail to validate understanding of the requirements, to ensure concurrence with stakeholder expectations, and to permit software development to begin.

(iii) Detail System-Wide Requirements

This task details one or more requirement that does not apply to a specific use case.

The purpose of this task is to describe one or more system-wide requirements (that could not be captured by use-cases and scenarios) in sufficient detail to validate understanding of the requirements, to ensure concurrence with stakeholder expectations, and to permit software development to begin.

(iv) Define Test Cases

This task requires development of the test cases and test data for the requirements to be tested.

The purpose of this task is to achieve a shared understanding of the specific conditions that the solution must meet.

8.3. Develop the Architecture

The students design, implement, test, integrate the solution for a number of critical use-case scenarios, and provide a demo of this solution.

The purpose of this activity is to propose a solution to meet the requirements of the system to be developed, and to prove a working prototype of the solution.

The students document architectural details as conformant to [Architecture Notebook Template](#), and provide a demo of the working prototype of their solution.

This activity includes the following tasks.

(i) Design the Solution

The students identify the elements and devise the interactions, behavior, relations, and data necessary to realize some functionality. The students render the design visually to aid in solving the problem and communicating the solution.

The purpose of this task is to describe the elements of the system so that they support the required behavior, are of high quality, and fit within the architecture.

(ii) Implement Developer Tests

The students implement one or more tests that enable the validation of the individual implementation elements through execution.

The purpose of this task is to prepare to validate an implementation element (e.g. an operation, a class, a stored procedure) through unit testing. The result is one or more new developer tests.

Developer testing is different from other forms of testing in that it is based on the expected behavior of code units rather than being directly based on the system requirements.

(iii) Implement Solution

The students implement source code to provide new functionality or fix defects.

The purpose of this task is to produce an implementation for part of the solution (such as a use-case scenario, a class, or component), or to fix one or more defects. The result is typically new or modified source code, which is referred to the implementation.

(iv) Run Developer Tests

The students run tests against the individual implementation elements to verify that their internal structures work as specified.

The purpose of this task is to verify that the implementation works as specified.

(v) Integrate and Create Build

This task describes how to integrate all changes made by developers into the code base and perform the minimal testing to validate the build.

The purpose of this task is to integrate all changes made by all developers into the code base and perform the minimal testing on the system increment in order to validate the build. The goal is to identify integration issues as soon as possible, so they can be corrected easily by the right person, at the right time.

(vi) Refine the Architecture

After performing the tasks above and implementing a working prototype of the architecture, the students may refine the architecture to an appropriate level of detail to support development.

8.4. Evolve the Design

After demonstrating a working architecture of the system and refining the architecture as defined in activity 8.3, the students evolve the architecture into a complete design of the system. The students document system design as conformant to [Software Design Description Template](#).

8.5. Implement the System

After proving a prototype of system architecture and defining the design, the students implement the system to meet entire set of system requirements including use case scenarios, system-wide requirements, and non-functional requirements. The students use

Software Requirements Specification and follow Software Design Description to carry out this activity.

The students may implement the system as a whole or in iterations. In case of iterative development, the project schedule announced for the delivery of working software iterations will be followed.

The students are expected to develop and follow a Coding Standard while implementing the system. Here is an example [Java Coding Standard](#). The students may readily utilize this standard or may define a new one according to their preferences.

8.6. Test the System

The students, from a system perspective, test and evaluate the developed requirements. The students develop test scripts for the selected test cases and document test scripts as conformant to [Test Script Template](#). The students run the tests as defined in the test scripts and document test results as conformant to [Software Test Report Template](#).

This activity includes the following tasks. If the students develop the system in iterations, they typically apply these tasks for each iteration.

(i) Implement Tests

The students implement Test Scripts to validate a Build of the solution. The students organize Test Scripts into suites, and collaborate to ensure appropriate depth and breadth of test feedback.

The purpose of this task is to implement step-by-step Test Scripts that demonstrate the solution satisfies the requirements. The students use Test Case Definition created by activity 8.2 (and delivered as an attachment to SRS) as input to this task.

(ii) Run Tests

The students run the appropriate tests scripts, analyze results, articulate issues, and document test results.

The purpose of this task is to provide feedback about how well a build satisfies the requirements.

8.7. Deliver the System

The students deliver a complete, defect-free system that meets all the requirements specified in Software Requirement Specification. The students provide source code and relevant data for the system in execution.

The acceptance of the final system is carried out by Teaching Assistants under the supervision of the Instructor. The assistants may run different tests, in addition to the ones specified in the Software Test Report, on the system during acceptance. It is recommended that students ensure defect-free execution of their systems prior to delivery.

9. References

- (i) IEEE std 830-1998 Recommended Practice for Software Requirements Specifications
- (ii) IEEE std 1016-1998 Recommended Practice for Software Design Descriptions
- (iii) IEEE std 829-1998 Standard for Software Test Documentation